

Lido

NEST

ENGAGEMENT

REVISION

DELIVERED

LEAD AUDITOR

SECURITY REVIEW

1.2

2026-07-24

Michal Převrátíl

Contents

1. Document Revisions.....	3
2. Overview.....	4
2.1. ack3.....	4
2.2. Review Methodology.....	5
2.3. Finding Classification	6
2.4. Review Team	8
2.5. Disclaimer	8
3. Executive Summary.....	9
Revision 1.0.....	9
Revision 1.1	12
Revision 1.2	13
4. Findings Summary	15
Report Revision 1.0	17
Revision Team	17
System Overview.....	17
Trust Model	18
Findings	20
Report Revision 1.1.....	62
Revision Team	62
Report Revision 1.2.....	63
Revision Team	63
Appendix A: How to cite.....	64

1. Document Revisions

1.0-draft	Draft Report	July 7, 2026
1.1	Final Report	July 17, 2026
1.2	Deployment Verification	July 24, 2026

2. Overview

This document presents the findings of our review.

2.1. ack3

[ack3](#) is an in-house team of security researchers performing audits focused on manual code reviews with extensive fuzz testing and AI-assisted workflows for Ethereum and Solana. The ack3 team is trusted by top-tier organizations in web3, including Lido, Safe, and Axelar.

We develop the open-source security and developer tooling [Wake](#), which powers the fuzzing, static analysis, and graph-based review on our audits and helps auditors discover hardly recognizable edge-case vulnerabilities.

Our team teaches blockchain security at the [Czech Technical University in Prague](#), led by founder Josef Gattermayer, Ph.D.

For ack3, security is a long-term partnership: we work alongside our clients from design through post-deployment.

2.2. Review Methodology

We begin each review by mapping the codebase and the interactions between contracts and external components using [Wake](#)'s graph tooling. We work with the provided documentation and materials to discover discrepancies between the expected behavior and the actual implementation. Where applicable, we build a mathematical model of the protocol to cross-compare with the code and identify subtle bugs and inconsistencies.

At the start of the review, we launch an automated [ack3](#) AI scan pinned to the scoped commits, running in parallel with the manual review. The scan provides additional coverage and new leads for the team to analyze; each scan finding is manually reviewed by the team.

During the review, we apply AI-assisted workflows and skills that support codebase onboarding and navigation, combined with Wake's static analyzer and language server for context-aware code analysis.

We launch AI agents to build on our vulnerability hypotheses and expand the attack surface with new escalation paths. All findings are manually reviewed and validated by the team. Where applicable, we confirm findings and verify the exact behavior of the code with unit tests and fuzzing. Where the engagement includes a fuzzing campaign, it is manually guided by an auditor; the fuzz tests themselves are AI-generated.

Once sufficient context is gathered, we search for deeper, chained issues based on the attack vectors discovered during review. This includes combining smaller weaknesses, testing trust-boundary assumptions, and evaluating whether unexpected behavior can lead to practical impact.

The validated results are documented in the report: findings with severity, affected components, impact, and recommended remediation.

2.3. Finding Classification

The Severity of each finding combines two sub-ratings, Impact and Likelihood, and ranges from Info to Critical.

If we find a scenario in which an issue is exploitable, it is assigned an impact rating of High, Medium, or Low, based on its consequences for the system. If we find no way to exploit it, or the issue is only exploitable given a change in configuration (system settings or parameters, such as deployment scripts, compiler configurations, or multi-signature wallets for owners) or in the codebase, it is assigned Warning or Info.

Low to High impact issues also have a Likelihood, which measures the probability of exploitation at runtime.

Severity

		LIKELIHOOD			
		HIGH	MEDIUM	LOW	N/A
IMPACT	HIGH	Critical	High	Medium	-
	MEDIUM	High	Medium	Low	-
	LOW	Medium	Low	Low	-
	WARNING	-	-	-	Warning
	INFO	-	-	-	Info

Table 1. Severity of findings

Impact

- **High** - Code that activates the issue leads to undefined or catastrophic consequences for the system.
- **Medium** - Code that activates the issue results in serious consequences.
- **Low** - Code that activates the issue has consequences that are recoverable or do not jeopardize the system's normal operation.
- **Warning** - The issue cannot be exploited given the current code or configuration, but could become a security vulnerability if either changed. If we have not found a way to exploit the issue given the time constraints, it may be marked as Warning or higher, based on our best estimate of whether it is currently exploitable.
- **Info** - The issue is on the borderline between code quality and security. Examples include insufficient logging for critical operations, or issues that would become security-related if code or configuration changed.

Likelihood

- **High** - The issue is exploitable by virtually anyone under any circumstance.
- **Medium** - Exploiting the issue requires non-trivial preconditions.
- **Low** - Exploiting the issue requires strict preconditions.

2.4. Review Team

For a revision’s authors, see the “Revision team” section in that revision’s chapter.

NAME	POSITION
Michal Převrátíl	Lead Auditor
Jan Kalivoda	Auditor
Naoki Yoshida	Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

2.5. Disclaimer

We have made our best effort to find all vulnerabilities in the system; however, our findings should not be considered a complete list of issues. Statements in this document should not be interpreted as investment or legal advice, nor should the authors be held accountable for decisions based on them.

3. Executive Summary

Lido NEST automates LDO buybacks funded by DAO treasury earnings, releasing funds only as fast as revenue accrues. Depending on its configured mode, it either deposits the acquired LDO alongside wstETH into a liquidity pool or sends it directly to the treasury. A keeper bot triggers the buybacks and deposits so the process runs with little manual intervention.

Revision 1.0

Lido engaged [ack3](#) to perform a security review of Lido NEST with a total time donation of 10 engineering days in a period between June 22 and July 7, 2026, with Michal Převrátíl as the lead reviewer.

The review was performed on commit [2e70a1b^{\[1\]}](#) in the [stonks](#) repository and commit [18ea278^{\[2\]}](#) in the [core](#) repository.

The review covered the following new files in the stonks repository:

- `contracts/automated-buybacks/BuybackExecutor.sol`
- `contracts/automated-buybacks/BuybackAllocator.sol`
- `contracts/automated-buybacks/revenue/StakingRevenueSource.sol`
- `contracts/automated-buybacks/AssetRecovererACL.sol`
- `contracts/automated-buybacks/revenue/RevenueSource.sol`
- `contracts/automated-buybacks/MathHelpers.sol`

The review also covered changes to the following files in the stonks repository since commit [4c6d195^{\[3\]}](#):

- `contracts/Stonks.sol`
- `contracts/factories/StonksFactory.sol`
- `contracts/Order.sol`

Finally, the review covered changes to `contracts/0.8.9/TokenRateNotifier.sol` in the core repository since commit [eae8614](#)^[4].

We began our review by understanding the overall architecture and the purpose of the system. Then, we performed a deep dive into the logic of the contracts and, in parallel, prepared a differential manually-guided fuzz test using the [Wake](#) testing framework. In the later phase of the review, we launched an automated Ack3 AI scan.

During the review, we paid special attention to:

- reviewing the integration with the Curve TwoCrypto pool, particularly the deposit protections;
- ensuring access controls are not too relaxed or too strict;
- assessing the permissionless functions for griefing vectors or ways to take advantage of the automated buyback system;
- identifying discrepancies between the states of the two contracts, such as an amount already budgeted on one side but not yet reflected on the other;
- examining the Chainlink integration and its consequences;
- confirming that permissioned roles retain full control over orders and can intervene in emergency scenarios; and
- looking for common issues such as data validation.

As part of the review, we evaluated the consequences of the absent slippage protection on Curve TwoCrypto pool deposits, under the stated requirement that Lido NEST be the only contract interacting with the pool and that no other interference from the Lido side is expected.

In our assessment, the current implementation is sound for the bootstrap phase and preferable to introducing a Chainlink-to-pool difference gate. We

nonetheless identified two residual risks:

- The Chainlink oracle may misreport the price of stETH or LDO, resulting in an imbalanced deposit and subsequent arbitrage.
- A malicious actor may poison the internal state of the pool (pool balances, price_scale), leading to higher imbalance fees paid by Lido NEST during deposits.

Both risks are inherent to the current design and, in our view, acceptable. The first stems from the Chainlink oracle itself and has a very low likelihood of occurring. For the second, the fees incurred by Lido NEST are bounded to a maximum of 0.026%, which is negligible relative to the deviation thresholds of the stETH/ETH (0.5%) and LDO/ETH (2%) oracles.

Conversely, introducing an EMA gate or any other Chainlink-to-pool difference gate would allow a malicious actor to temporarily poison the shallower pool and block deposits through the gate.

The review was conducted against the following pool parameters:

PARAMETER	VALUE
A	400000
gamma	1450000000000000
mid_fee	500000
out_fee	5000000
fee_gamma	2300000000000000
allowed_extra_profit	20000000000000
adjustment_step	1460000000000000

PARAMETER	VALUE
ma_exp_time	866

Our review resulted in 16 findings, ranging from Info to Low severity. Most of the low-severity findings are edge cases in how the system tracks state across time or across cooperating contracts, rather than directly exploitable vulnerabilities. For instance, [L9](#) stems from a status enum that drifted out of sync with its interface, [L4](#) shows a fixed-window rate limiter that can be straddled to briefly exceed its cap, and [L3](#) leaves funds unmonitored after a configuration change. We recommend addressing these to keep the system's accounting and monitoring consistent under all conditions.

The fuzz tests created during the review are available at [ack3-ai/tests-lido-nest](https://github.com/ack3-ai/tests-lido-nest).

[ack3](#) recommends Lido:

- ensure the maximum acceptable price deviation for stETH-to-LD0 swaps aligns with the Chainlink oracle's thresholds;
- always verify the deployed Stonks address and confirm it holds the expected configuration parameters;
- rework the revenue source removal logic to prevent denial of service;
- ensure the EMA gate is not so strict that it becomes grievable or causes denial of service without malicious intent;
- read and review the complete report; and
- address all identified issues.

See [Report Revision 1.0](#) for the system overview and trust model.

Revision 1.1

Lido engaged [ack3](#) to perform a fix review for the findings in the previous

revision. The review was performed on commit [10a2499^{\[5\]}](#) in the [stonks](#) repository and commit [dc9066b^{\[6\]}](#) in the [core](#) repository. Beyond the changes addressing the reported findings, these commits also introduced minor code and comment modifications, which were likewise covered by the review.

Of the 16 findings from the previous revision, 8 were fixed and the remaining 8 were acknowledged. No new findings were identified during the review.

Revision 1.2

Lido engaged [ack3](#) to perform deployment verification of Lido NEST on the Ethereum mainnet. The verification was performed on the same commits as the previous revision: [10a2499^{\[7\]}](#) in the [stonks](#) repository and [dc9066b^{\[8\]}](#) in the [core](#) repository.

The verification concluded successfully with a full match of the runtime bytecode, excluding the metadata hash, and reasonable values used for constructor arguments. Deployment parameters were additionally checked against [LIP-36](#) for the parameters it specifies.

The following contracts were verified:

- StonksFactory: [0x632C0CCDca849eeD780FC685BBa9AbC3c7407Cb2](#)
- Order (sample): [0x2569633AdB492ca9327cb7433277aa7c68D69e28](#)
- StakingRevenueSource:
[0x6220212a33a87Ed7Cc386B67eB2c393974F28C38](#)
- BuybackExecutor: [0x6c213ca5A10Cc26548C742229569B4AeD2A9C9B7](#)
- BuybackAllocator: [0xAA568141c051f2D1132b110f8391F18D48E8D889](#)
- Stonks (LP mode): [0x8c595aA4AEc6F42B9e7D77F83179768D37CE3042](#)
- Stonks (Treasury mode):
[0xb368586CB980895E51e1D82102E63b3F69d3F151](#)
- TokenRateNotifier: [0xbe05d12Fd10919F1881125006523452F6aFF791b](#)

- LidoLocator: [0xF2Ffb952e129a63F0614Ff87126E1d4a494A2313](#)

The deployment verification script is available at [ack3-ai/tests-lido-nest](#).

[1] full commit hash: 2e70a1b64b951fd5ba627794c040aab1d37ed77e, link to [commit](#)

[2] full commit hash: 18ea278ed485cc4c3b82fce3a41561acaa61005a, link to [commit](#)

[3] full commit hash: 4c6d1955268b8c97df5fd93ea8912878e07a1252, link to [commit](#)

[4] full commit hash: eae861480b7bca5a1a69a29de85cab6af3de1e01, link to [commit](#)

[5] full commit hash: 10a249971099be1b36c27f43a3f0a520389fb62e, link to [commit](#)

[6] full commit hash: dc9066b6f9b5c6c3b65f4a4aac338427198f0e98, link to [commit](#)

[7] full commit hash: 10a249971099be1b36c27f43a3f0a520389fb62e, link to [commit](#)

[8] full commit hash: dc9066b6f9b5c6c3b65f4a4aac338427198f0e98, link to [commit](#)

4. Findings Summary

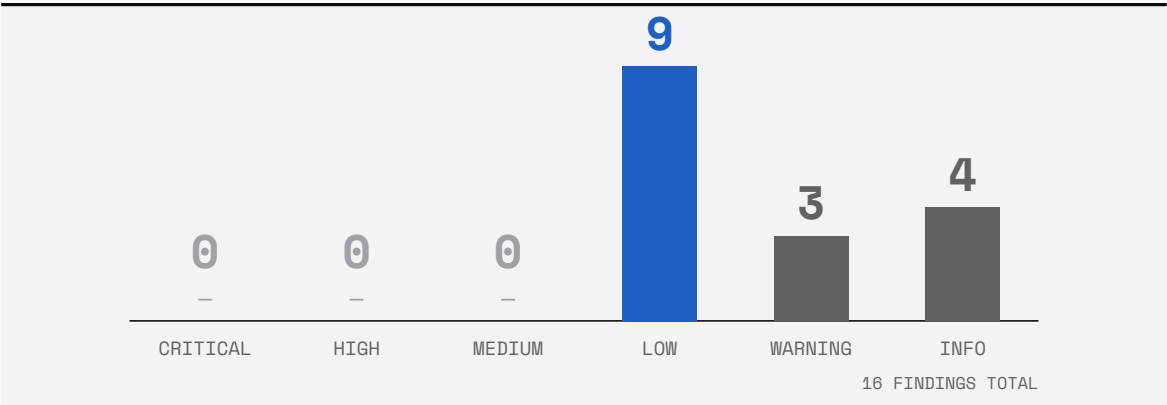


Figure 1. Findings by Severity

FINDING TITLE	SEVERITY	REVISION	STATUS
L1: Timed price mismatch in buyback budget	LOW	1.0	Acknowledged
L2: Failed revenue source recovery	LOW	1.0	Fixed
L3: Incomplete Stonks switch wind-down	LOW	1.0	Fixed
L4: Daily spend cap straddling	LOW	1.0	Acknowledged
L5: Unconverted Revenue Excluded From Baselines	LOW	1.0	Acknowledged
L6: Empty revert data breaks observer isolation	LOW	1.0	Acknowledged
L7: Recovered revenue reweighted by increased surplus share	LOW	1.0	Fixed

FINDING TITLE	SEVERITY	REVISION	STATUS
L8: Unfillable buyback order slot	LOW	1.0	Fixed
L9: AllocationStatus enum desynchronization	LOW	1.0	Fixed
W1: Valuation-blind LDO buyback cadence	WARNING	1.0	Acknowledged
W2: Chainlink phase upgrade price-read DoS	WARNING	1.0	Acknowledged
W3: Front-runnable precomputed Stonks address	WARNING	1.0	Acknowledged
I1: Treasury revenue undercount from per-module rounding dust	INFO	1.0	Acknowledged
I2: Staking router fee return type mismatch	INFO	1.0	Fixed
I3: Ignored slippage floors on full LP withdrawals	INFO	1.0	Fixed
I4: Missing interface inheritance	INFO	1.0	Fixed

Table 2. Table of Findings

Report Revision 1.0

Revision Team

NAME	POSITION
Michal Převrátíl	Lead Auditor
Jan Kalivoda	Auditor
Naoki Yoshida	Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

System Overview

NEST (Automated Buyback System) is a revenue-paced system that turns DAO-owned stETH into LDO under verifiable on-chain conditions. Two core contracts work together.

The **BuybackAllocator** holds the stETH pot and releases it to a single executor only as fast as the protocol earns revenue; it tracks a running signed `budgetUSD` that each call updates by a share of new revenue minus a daily reserve, spending only the positive part within per-day and per-year caps.

The **BuybackExecutor** receives that stETH (through the `onStEthAllocated` hook) plus the LDO from settlements. It either sells stETH for LDO over the Stonks/CoW Swap path via a permissionless `placeOrder`, or, in LP mode, pairs bought LDO with wstETH into a Curve TwoCrypto pool via `addLiquidity`—protected by an EMA-versus-oracle gate and per-deposit USD limits. The operating mode follows the active Stonks receiver: LP mode settles LDO to the executor, treasury mode to the treasury.

Revenue enters through the **RevenueSource** base and its concrete

StakingRevenueSource, which captures DAO staking revenue from each Lido rebase (via the `TokenRateNotifier` callback) and converts it to USD through the `OracleRouter`. Both new contracts share the **AssetRecovererACL** base for roles, reentrancy protection, and treasury recovery, along with a common **MathHelpers** library.

Finally, the in-scope updates to **Stonks**, **Order**, **StonksFactory**, and **TokenRateNotifier** add a configurable settlement receiver and an argument-bearing observer channel forwarding the full rebase data, enabling the executor's dual-mode routing and simpler on-chain revenue tracking.

Trust Model

The work functions are permissionless (built for a keeper bot), but every economic parameter, contract wiring, and asset-recovery path is gated behind DAO governance. NEST fully trusts governance and the external price/settle/rebase systems; it does not trust the keepers, the CoW Swap solvers, or the public caller beyond what the on-chain checks enforce.

Trusted actors (privileged roles)

- **Admin / governance (DEFAULT_ADMIN_ROLE, the DAO Agent)**. Fully trusted and the root of the model. It performs the one-time `activate()`, runs every parameter setter, calls `setExecutor` and `setStonksAndOperatingMode`, and registers or removes revenue sources. A malicious or careless admin can misprice the whole system (caps, tolerances, `poolBootstrapMinTv1Usd`, deposit bounds), point the executor at a hostile **Stonks**, or wire in a bad revenue source. This is accepted, standard DAO trust.
- **MANAGER_ROLE**. Trusted to recover ETH/ERC-20 to the treasury and to call `removeLiquidityAndRecoverToTreasury`, which burns LP, unwraps `wstETH`, and returns both coins to the treasury. Funds only ever move to the immutable **TREASURY**, so this role can force an exit but cannot redirect

value.

- **EMERGENCY_ROLE.** Trusted to trigger an independent pause and forward pause calls to the active Stonks. It can halt but not move funds.
- **ALLOCATOR_ROLE.** Held by the BuybackAllocator contract, not a human. It is the only caller allowed to invoke the executor's `onStEthAllocated` hook, which binds the two contracts together as a single trust boundary.

Untrusted / bounded actors

- **Keepers and the general public.** `allocate()`, `placeOrder`, `addLiquidity`, and `convertPendingRevenueToUSD` are permissionless. The system does not trust the caller: correctness rests on the checkpoint math, the per-day/per-year caps, the oracle-derived order floor, and the EMA-vs-oracle deposit gate. The keeper is relied on only for **liveness** (timely calls), never for safety.
- **CoW Swap solvers.** Untrusted counterparties. Protection comes from the per-unit oracle floor re-validated on every fill and from the hardwired settlement receiver, so a solver cannot fill below the floor or divert output.

Trusted external dependencies (out of scope, assumed correct)

- **OracleRouter and its Chainlink feeds** — assumed accurate, live, and within their staleness/deviation bounds.
- **Stonks trade-execution infrastructure and CoW Swap / GPv2Settlement** — assumed to settle honestly at or above the derived floor.
- **Lido core** (`stETH`, `wstETH`, `LidoLocator`, `StakingRouter`, rebase mechanism, `TokenRateNotifier`) — assumed to deliver each rebase payload faithfully, once per report, to the correctly wired `postTokenRebaseReceiver`.
- **Curve TwoCrypto LDO/wstETH pool and its EMA oracle** — assumed

manipulation- resistant enough to serve as the deposit divergence reference.

Findings

This section lists the findings from this revision; for all findings, see the [Findings Summary](#).

L1: Timed price mismatch in buyback budget

LOW SEVERITY ISSUE

Impact:	Low	Likelihood:	Medium
Type:	Logic error	Discoverer:	Human
Target:	StakingRevenueSource.sol, BuybackAllocator.sol		

Description

StakingRevenueSource.convertPendingRevenueToUSD() records a fixed stETH revenue amount in USD using the spot price at the permissionless conversion call. BuybackAllocator.allocate() later converts that USD budget back to stETH using a separately timed spot price, with only a lower price floor.

Listing 1. Excerpt from [StakingRevenueSource.convertPendingRevenueToUSD](#)

```

202 address stEth = LIDO_LOCATOR.lido();
203 (uint256 stEthUsdPrice, ) = ORACLE_ROUTER.getUsdPrices(stEth, stEth);
204
205 if (stEthUsdPrice == 0) {
206     revert OracleReturnedZeroPrice();
207 }
208
209 uint256 revenueUSD = (pending * stEthUsdPrice) / PRICE_UNIT;
```

Listing 2. Excerpt from [BuybackAllocator.allocate](#)

```

682 spendableStEth = Math.mulDiv(spendableUSD, PRICE_UNIT, stEthPriceUSD);
683 spendableStEth = Math.min(spendableStEth, STETH.balanceOf(address(this)));
684 spendableUSD = Math.mulDiv(spendableStEth, stEthPriceUSD, PRICE_UNIT);
```

Because the allocator spends from a DAO stETH principal pot, the USD budget does not bound the real stETH outflow. A caller can convert pending revenue when the spot price is high and allocate when the spot price is lower, causing more stETH principal to leave per unit of earned revenue while still respecting

the nominal daily and yearly USD caps.

Exploit scenario

Alice monitors the price source used by the revenue source and allocator. When the spot stETH price is locally high, she calls `convertPendingRevenueToUSD()` to maximize Bob's recorded USD surplus for the same pending stETH revenue. Later, when the spot price is lower but still above `minStEthPriceUSD`, Alice calls `allocate()`. The allocator spends the inflated USD budget at the lower price, so Bob's DAO pot releases excess stETH even though the USD caps are not exceeded.

Recommendation

Use a common bounded price for both conversion legs, such as a shared TWAP with max-staleness and max-divergence checks. Alternatively, denominate budgets and daily/yearly caps in stETH, or otherwise cap the actual stETH amount released from the DAO pot.

Acknowledgment 1.1

The finding was acknowledged; no smart contract changes were made. According to the client, both conversion legs price at the oracle spot — which the client considers non-manipulable — and the allocated stETH is sold at the same allocation-time price, so that the USD value released equals the budget spent. The client intends to address the residual timing discretion operationally, updating the NEST handbook keeper duties to prompt a post-rebase call to `convertPendingRevenueToUSD()`, with a corresponding entry in the governance runbook.

[Go back to Findings Summary](#)

L2: Failed revenue source recovery

LOW SEVERITY ISSUE

Impact:	Low	Likelihood:	Low
Type:	Logic error	Discoverer:	Human
Target:	BuybackAllocator.sol		

Description

BuybackAllocator stores only the aggregate lastTotalRevenueUSD and does not persist each source's last successful cumulative value. As a result, failed revenue sources cannot be isolated from checkpointing, activation, or removal.

_checkpoint() reads the current aggregate through _revenueSumUSD(), derives the budget delta, and then commits the aggregate as the new baseline. When a previously working source starts reverting permanently, _revenueSumUSD() scores it as 0 while lastTotalRevenueUSD still includes that source's last-good history. This can reduce already-earned budgetUSD and force surviving sources to re-earn the missing history before allocations resume.

Listing 3. Excerpt from [BuybackAllocator._checkpoint](#)

```
452 function _checkpoint() internal {
453     (int256 budgetDeltaUSD, uint256 totalRevenueUSD, uint256 reserveUSD) =
        _budgetable();
454
455     budgetUSD += budgetDeltaUSD;
456     lastTotalRevenueUSD = totalRevenueUSD;
457     // Move the reserve cursor to the next day.
458     reserveAnchorTS = _nextDayStartTS();
459     emit ReserveAnchored(reserveAnchorTS);
460
461     emit Checkpoint(lastTotalRevenueUSD, reserveUSD, budgetDeltaUSD,
        budgetUSD);
462 }
```

Listing 4. Excerpt from [BuybackAllocator._budgetable](#)

```
709     totalRevenueUSD = _revenueSumUSD();
710     reserveUSD = _reserveCurrentUSD();
711     int256 surplusUSD = int256(totalRevenueUSD) -
712         int256(lastTotalRevenueUSD) -
713         int256(reserveUSD);
714     budgetDeltaUSD = (surplusUSD * int256(uint256(surplusShareBP))) /
715         int256(MAX_BASIS_POINTS);
715 }
```

Listing 5. Excerpt from [BuybackAllocator._revenueSumUSD](#)

```
781 function _revenueSumUSD() internal view returns (uint256 revenueSumUSD) {
782     address[] memory sources = _revenueSources.values();
783     for (uint256 i; i < sources.length; ++i) {
784         try IRevenueSource(sources[i]).getCumulativeRevenueUSD() returns
785             (uint256 revenue) {
786             revenueSumUSD += revenue;
787         } catch {}
788     }
```

The tolerant read is also incomplete. A codeless source or a source returning malformed data can fail during ABI decoding in the allocator frame, which is not handled by the typed try/catch and can make checkpointing revert instead of scoring the source as 0.

Removal has the same recovery gap. `removeRevenueSource()` checkpoints first, and `_removeRevenueSource()` strictly re-reads the removed source to adjust `lastTotalRevenueUSD`. A failed source can therefore block its own removal either during the checkpoint or during the strict re-read.

Listing 6. Excerpt from [BuybackAllocator.removeRevenueSource](#)

```
410 function removeRevenueSource(
411     address source_
412 ) external onlyRole(DEFAULT_ADMIN_ROLE) whenActivated {
413     _checkpoint();
414     _removeRevenueSource(source_);
```



```
415 }
```

Listing 7. Excerpt from [BuybackAllocator.removeRevenueSource](#)

```
522 function _removeRevenueSource(address source_) internal {
523     if (!_revenueSources.remove(source_)) {
524         revert RevenueSourceNotRegistered();
525     }
526
527     // removeRevenueSource() checkpoints first, so lastTotalRevenueUSD
already includes this
528     // source's current total – the subtraction is therefore exact and
cannot underflow.
529     lastTotalRevenueUSD -=
        IRevenueSource(source_).getCumulativeRevenueUSD();
530
531     emit RevenueSourceRemoved(source_);
532 }
```

The same lack of per-source last-good values affects pre-activation recovery because activation requires readable sources to establish the baseline while source-set changes are activation-gated. The launch `StakingRevenueSource` is storage-backed and non-upgradeable, so this is mainly a risk once external-dependency, proxy, or upgradeable revenue sources are added.

Exploit scenario

Alice deploys an upgradeable revenue source and the admin registers it in `BuybackAllocator`. The allocator activates and the source accumulates revenue.

A later source upgrade breaks `getCumulativeRevenueUSD()` or makes the source return invalid data. Bob calls `allocate()`. The checkpoint either treats Alice's source as zero and reduces the budget against the old aggregate baseline, or reverts while decoding the source response.

The admin then tries to remove Alice's failed source. Removal also depends on checkpointing or on a strict read from the failed source, so the source cannot

be removed on-chain and buybacks remain halted or underfunded until the source is repaired or the allocator is redeployed.

Recommendation

Store the latest successful cumulative revenue value for each source. Use those stored values when checkpointing, activating, and removing sources so a failed source remains at its last-good value instead of being zeroed or re-read strictly.

Implement source reads with low-level `staticcall` and validate that the source has code, the call succeeds, and the `returndata` is exactly one `uint256` before decoding. Allow forced removal to subtract the stored last-good value for the removed source, including during pre-activation recovery.

Fix 1.1

The finding was fixed by treating revenue sources as trusted. Because trusted sources are now assumed to always return valid, non-reverting data, the failed-source checkpointing, removal, and recovery scenarios described above no longer apply.

[Go back to Findings Summary](#)

L3: Incomplete Stonks switch wind-down

LOW SEVERITY ISSUE

Impact:	Low	Likelihood:	Low
Type:	Logic error	Discoverer:	Human
Target:	BuybackExecutor.sol		

Description

`_setStonksAndOperatingMode()` replaces the active stonks instance and sweeps only the tracked order. It does not handle loose `stETH` left on the previous Stonks, so that balance drops out of the executor's automated placement and monitoring after the pointer is changed.

Listing 8. Excerpt from [BuybackExecutor._setStonksAndOperatingMode](#)

```
881 bool previousLpModeEnabled = lpModeEnabled;
882 address previousStonks = address(stonks);
883
884 lpModeEnabled = lpModeEnabled_;
885 stonks = IStonks(stonks_);
886 stonksOrderDurationSeconds =
    IStonks(stonks_).ORDER_DURATION_IN_SECONDS().toUint32();
```

The same switch checklist also misses the LP-to-treasury wind-down case.

`_computeLpModeFreeStEth()` reserves `stETH` against executor-held LDO in LP mode, but treasury mode sells the executor's free `stETH`; the first post-switch allocation can therefore sell the former pairing reserve while the held LDO remains on the executor until manual recovery.

Listing 9. Excerpt from [BuybackExecutor.onStEthAllocated](#)

```
459 uint256 freeStEth = lpMode ? _computeLpModeFreeStEth() :
    STETH.balanceOf(address(this));
```

```
460 uint256 stEthAmountToSell = lpMode ? freeStEth / 2 : freeStEth;
```

The current dev note warns operators only about abandoned orders, not residual Stonks balances or the executor-held LD0 reserve.

Listing 10. Excerpt from [BuybackExecutor._setStonksAndOperatingMode](#)

```
844 * @dev    The new Stonks must sell stETH for LD0 with this contract as its
            manager. Switching
845 *          disconnects this contract from the previous Stonks. An expired
            tracked order is swept
846 *          and its residual recovered to the previous Stonks. A live order
            survives the sweep and
847 *          is abandoned, recorded by `OrderAbandoned`; anyone can call its
            `recoverTokenFrom`
848 *          after expiry to return the stETH to the previous Stonks. In LP
            mode an abandoned order
849 *          no longer reserves stETH against new allocations, so pause
            allocations and recover it
850 *          before switching.
```

As a result, routine Stonks replacement can silently require out-of-band governance or manager recovery even when operators follow the documented checklist.

Exploit scenario

Alice replaces the active Stonks while the previous Stonks has idle stETH that is not part of `lastOrderAddress`. The executor points to the new Stonks and sweeps only the tracked order, so the previous Stonks' idle stETH is no longer placed or monitored by the automated buyback flow.

Alice also performs an LP-to-treasury switch while the executor holds LD0 accumulated for LP deposits. On the next allocation, the executor forwards the formerly reserved pairing stETH for sale, and the LD0 remains on the executor until a separate `recoverERC20()` transaction.

Recommendation

Handle all residual balances during `_setStonksAndOperatingMode()`. Drain or transfer the previous Stonks' idle `stETH` before updating the active pointer, and explicitly process executor-held `LDO` when switching from LP mode to treasury mode. At minimum, extend the dev note and operator runbook to require these recovery steps before switching.

Fix 1.1

The finding was fixed by pulling the residual `stETH` from the previous Stonks during the switch. The dev note was also updated accordingly.

[Go back to Findings Summary](#)

L4: Daily spend cap straddling

LOW SEVERITY ISSUE

Impact:	Low	Likelihood:	Medium
Type:	Logic error	Discoverer:	Human
Target:	BuybackAllocator.sol		

Description

The protocol enforces a daily USD spend cap via a fixed-window rate limiter. Because the window is reset immediately at a deterministic boundary, spend can be “straddled” across the boundary to release up to 2x the daily cap within a short time.

`_windowSpent` treats the spent amount as 0 as soon as `block.timestamp` reaches `endTS`:

Listing 11. Excerpt from [BuybackAllocator._windowSpent](#)

```
730 * @param window_ Spend window to read.
731 * @return spent USD spent in the current window.
732 */
733 function _windowSpent(SpendWindow storage window_) internal view returns
    (uint256 spent) {
734     spent = block.timestamp >= window_.endTS ? 0 :
        uint256(window_.spentUSD);
735 }
736
737 /**
```

`_rollWindow` then resets `spentUSD` to 0 when the boundary is crossed:

Listing 12. Excerpt from [BuybackAllocator._rollWindow](#)

```
476 uint192 spent = window_.spentUSD;
477 if (block.timestamp >= window_.endTS) {
```

```
478     uint64 newEndTS = uint64(
479         activationTS +
480         ((block.timestamp - activationTS) / windowDuration_ + 1) *
481         windowDuration_
482     );
483
484     emit WindowRolled(windowDuration_, newEndTS, spent);
485
486     window_.endTS = newEndTS;
487     spent = 0;
488 }
```

Since `allocate()` is permissionless, a caller can deliberately place one call just before the daily window ends and another just after it begins. This defeats the pacing/market-impact goal of the daily cap.

Exploit scenario

Alice monitors the upcoming daily window boundary.

Just before the window ends, Alice calls `allocate()` to spend the remaining daily cap. Immediately after the boundary, Alice calls `allocate()` again and spends up to the full daily cap for the new window.

As a result, up to 2x the intended daily outflow can occur within a short time, provided sufficient budget and buyback funds are available.

Recommendation

Replace the fixed-window limiter with a sliding-window limiter. Alternatively, carry over a portion of the spent amount into the next window so near-boundary spends do not reset to a full daily cap immediately.

Acknowledgment 1.1

The finding was acknowledged. The client noted that the yearly cap bounds total spending, and that sale pacing is enforced by the executor holding only a single live order at a time, itself bounded by `maxAllowedOrderAmount`. A

clarifying code comment was added.

[Go back to Findings Summary](#)

L5: Unconverted Revenue Excluded From Baselines

LOW SEVERITY ISSUE

Impact:	Low	Likelihood:	Medium
Type:	Logic error	Discoverer:	Human
Target:	StakingRevenueSource.sol, BuybackAllocator.sol		

Description

StakingRevenueSource records treasury fees from rebases in pendingRevenueStEth before they are converted to USD revenue. The pending amount is not included in getCumulativeRevenueUSD() until convertPendingRevenueToUSD() is called.

Listing 13. Excerpt from [StakingRevenueSource.pushTokenRate](#)

```
177 uint256 newPending = pendingRevenueStEth + treasuryStEth;
178 pendingRevenueStEth = newPending;
```

Listing 14. Excerpt from [StakingRevenueSource.convertPendingRevenueToUSD](#)

```
194 function convertPendingRevenueToUSD() external {
195     uint256 pending = pendingRevenueStEth;
196     if (pending == 0) {
197         return;
198     }
199
200     pendingRevenueStEth = 0;
```

BuybackAllocator uses the converted cumulative revenue value as the baseline in both activate() and addRevenueSource().

Listing 15. Excerpt from [BuybackAllocator.activate](#)

```
279 lastTotalRevenueUSD = _revenueSumStrictUSD();
```

Listing 16. Excerpt from [BuybackAllocator.addRevenueSource](#)

```
510 lastTotalRevenueUSD += IRevenueSource(source_).getCumulativeRevenueUSD();
```

If a source has non-zero pendingRevenueStEth when the baseline is recorded, that pre-baseline revenue is omitted. When the pending stETH is later converted, it increases cumulative revenue above the baseline and the allocator treats surplusShareBP of that value as new surplus. This can over-bank the buyback budget with revenue that the baseline was intended to exclude.

Exploit scenario

Alice operates an active BuybackAllocator. She adds a StakingRevenueSource that has already accrued unconverted pendingRevenueStEth from earlier rebases. The allocator records only the source's converted cumulative USD revenue as the baseline.

Bob calls the permissionless convertPendingRevenueToUSD(). At the next checkpoint, the allocator sees the converted pre-baseline pending amount as post-baseline surplus and increases the buyback budget. The DAO spends more on buybacks than the baseline policy permits.

Recommendation

Settle or value pending revenue before recording baselines. For example, make activate() and addRevenueSource() call convertPendingRevenueToUSD() on each affected source, or add a baseline accessor that returns converted revenue plus the current USD value of pendingRevenueStEth.

Acknowledgment 1.1

The finding was acknowledged; no functional smart contract changes were made beyond a clarifying code comment. The client intends to address it operationally; the activation vote will include an item to run

convertPendingRevenueToUSD() on each revenue source before activate() is called, so that pending revenue is settled into the recorded baseline. This procedure will also be documented in the NEST handbook and the governance runbook.

[Go back to Findings Summary](#)

L6: Empty revert data breaks observer isolation

LOW SEVERITY ISSUE

Impact:	Low	Likelihood:	Low
Type:	Denial of service	Discoverer:	Human
Target:	TokenRateNotifier.sol, Accounting.sol, StakingRevenueSource.sol		

Description

TokenRateNotifier.handlePostTokenRebase attempts to notify each observer inside a per-observer try/catch. A failing observer is intended to only emit PushTokenRateFailed and allow the loop to continue.

However, TokenRateNotifier._handlePushFailure re-reverts when the caught revert data is empty. This re-revert happens outside of the try/catch and therefore reverts the entire handlePostTokenRebase call.

Listing 17. Excerpt from [TokenRateNotifier](#)

```
134 if (entry.kind == ObserverKind.NoArgs) {
135     // solhint-disable-next-line no-empty-blocks
136     try ITokenRatePusher(observerAddr).pushTokenRate() {}
137     catch (bytes memory lowLevelRevertData) {
138         _handlePushFailure(observerAddr, lowLevelRevertData);
139     }
```

Listing 18. Excerpt from [TokenRateNotifier](#)

```
171 function _handlePushFailure(address observer_, bytes memory
    lowLevelRevertData_) internal {
172     if (lowLevelRevertData_.length == 0) revert
        ErrorTokenRateNotifierRevertedWithNoData();
173     emit PushTokenRateFailed(observer_, lowLevelRevertData_);
174 }
```

Empty revert data is not a reliable out-of-gas signal. It can also be produced by a bare `revert()` or `require(false)` without a reason string, or by an ABI decoder error.

Because `Accounting._notifyRebaseObserver` calls `handlePostTokenRebase` without its own `try/catch`, the empty-data re-revert can bubble up and revert the entire oracle report execution.

Listing 19. Excerpt from [Accounting](#)

```
480 if (address(_postTokenRebaseReceiver) != address(0)) {
481     _postTokenRebaseReceiver.handlePostTokenRebase(
482         _report.timestamp,
483         _report.timeElapsed,
484         _pre.totalShares,
485         _pre.totalPooledEther,
486         _update.postTotalShares,
487         _update.postTotalPooledEther,
488         _update.sharesToMintAsFees
489     );
490 }
```

Additionally, `StakingRevenueSource.pushTokenRate` NatSpec states that reverts are non-blocking due to the notifier wrapping the call in `try/catch`. This guarantee is false for empty-data failures, which are treated as fatal by the notifier.

Listing 20. Excerpt from [StakingRevenueSource](#)

```
112 * @dev Authorization is checked live against
    `LIDO_LOCATOR.postTokenRebaseReceiver()` so
113 * a locator upgrade that retargets the receiver is auto-followed.
    The notifier
114 * wraps this call in try/catch, so reverts here are non-blocking
    and surface as
115 * `PushTokenRateFailed` on the notifier. The rebase critical path
    is intentionally
```

Exploit scenario

Alice (governance) registers an observer that forwards to an address without code, or reverts without a reason string.

Bob submits an oracle report that reaches `Accounting._notifyRebaseObserver`. During `TokenRateNotifier.handlePostTokenRebase`, the observer reverts with empty data.

`TokenRateNotifier._handlePushFailure` re-reverts with `ErrorTokenRateNotifierRevertedWithNoData()`. This revert bubbles up through `Accounting._notifyRebaseObserver` and reverts the entire oracle report, blocking rebases until governance removes the observer.

Recommendation

Do not treat `data.length == 0` as sufficient evidence of out-of-gas.

Snapshot `gasleft()` before the observer call. In the catch, only bubble the error when the remaining gas is near the EIP-150 1/64 retained-gas floor. Otherwise, emit `PushTokenRateFailed` and continue.

Update `StakingRevenueSource.pushTokenRate NatSpec` to document the empty-data invariant, or adjust the notifier to match the stated guarantee.

Acknowledgment 1.1

The finding was acknowledged. The client noted that observers are trusted and are always guaranteed to revert with non-zero data, leaving out-of-gas as the only remaining source of empty-data reverts.

[Go back to Findings Summary](#)

L7: Recovered revenue reweighted by increased surplus share

LOW SEVERITY ISSUE

Impact:	Low	Likelihood:	Low
Type:	Logic error	Discoverer:	Human
Target:	BuybackAllocator.sol		

Description

setSurplusShareBP() checkpoints revenue while the old surplusShareBP is still active, and only then stores the new share.

Listing 21. Excerpt from [BuybackAllocator](#)

```
327 function setSurplusShareBP(  
328     uint16 surplusShareBP_  
329 ) external onlyRole(DEFAULT_ADMIN_ROLE) whenActivated {  
330     _checkpoint();  
331     _setSurplusShareBP(surplusShareBP_);  
332 }
```

The checkpoint persists lastTotalRevenueUSD from the tolerant revenue sum. If a source reverts, that source contributes zero to the persisted baseline.

Listing 22. Excerpt from [BuybackAllocator](#)

```
452 function _checkpoint() internal {  
453     (int256 budgetDeltaUSD, uint256 totalRevenueUSD, uint256 reserveUSD) =  
454         _budgetable();  
455     budgetUSD += budgetDeltaUSD;  
456     lastTotalRevenueUSD = totalRevenueUSD;  
457     // Move the reserve cursor to the next day.  
458     reserveAnchorTS = _nextDayStartTS();  
459     emit ReserveAnchored(reserveAnchorTS);  
460  
461     emit Checkpoint(lastTotalRevenueUSD, reserveUSD, budgetDeltaUSD,
```

```

    budgetUSD);
462 }

```

Listing 23. Excerpt from [BuybackAllocator](#)

```

781 function _revenueSumUSD() internal view returns (uint256 revenueSumUSD) {
782     address[] memory sources = _revenueSources.values();
783     for (uint256 i; i < sources.length; ++i) {
784         try IRevenueSource(sources[i]).getCumulativeRevenueUSD() returns
            (uint256 revenue) {
785             revenueSumUSD += revenue;
786         } catch {}
787     }
788 }

```

`_budgetable()` applies the current `surplusShareBP` to the difference between the current revenue and the persisted baseline.

Listing 24. Excerpt from [BuybackAllocator](#)

```

704 function _budgetable()
705     internal
706     view
707     returns (int256 budgetDeltaUSD, uint256 totalRevenueUSD, uint256
        reserveUSD)
708 {
709     totalRevenueUSD = _revenueSumUSD();
710     reserveUSD = _reserveCurrentUSD();
711     int256 surplusUSD = int256(totalRevenueUSD) -
712         int256(lastTotalRevenueUSD) -
713         int256(reserveUSD);
714     budgetDeltaUSD = (surplusUSD * int256(uint256(surplusShareBP))) /
        int256(MAX_BASIS_POINTS);
715 }

```

At a constant share, a transient source revert creates a negative delta and the later recovery creates an equal positive delta, so the budget self-heals. When the share is increased between those checkpoints, the negative delta is banked at the old share, the false-low baseline is persisted, and the recovered cumulative revenue is banked at the new higher share. This can overstate `budgetUSD` and cause extra `stETH` to be allocated, subject only to

the daily and yearly caps.

Exploit scenario

Alice deploys the allocator with one revenue source, `reserveDailyRateUSD = 0`, activation baseline `1,000,000`, and `surplusShareBP = 50%`. The source later has true cumulative revenue of `1,600,000`.

Alice raises `surplusShareBP` to `100%` while the source is transiently reverting. The checkpoint sees tolerant revenue of `0`, banks $(0 - 1,000,000) * 50\% = -500,000$, and stores `lastTotalRevenueUSD = 0`.

The source recovers, and Bob calls `allocate()`. The next checkpoint sees `1,600,000` and banks $1,600,000 * 100\% = 1,600,000$, so `budgetUSD` becomes `1,100,000`. Without the transient revert, only the `600,000` revenue earned before the share change would have been banked at `50%`, leaving `300,000` of budget. The over-bank is `800,000`; at a `2,000 USD stETH` price, this allows an extra `400 stETH` to leave the allocator.

Recommendation

Make the pre-change checkpoint in `setSurplusShareBP()` use a strict revenue read, so the share change reverts while any source is unreadable.

Alternatively, prevent tolerant-low baselines from crossing a share change, for example by snapshotting enough state to weight recovered revenue under the share that was active when it accrued.

Fix 1.1

The finding was fixed by marking revenue sources as trusted and removing the non-strict `_revenueSumUSD` variant. As a result, a transient revenue source revert can no longer affect the contract's internal accounting.

[Go back to Findings Summary](#)

L8: Unfillable buyback order slot

LOW SEVERITY ISSUE

Impact:	Low	Likelihood:	Low
Type:	Denial of service	Discoverer:	Human
Target:	BuybackExecutor.sol, Order.sol		

Description

BuybackExecutor.placeOrder is permissionless, but it tracks only one live order through lastOrderAddress and rejects replacement while that pointer is set. After a placement, the executor records the order and its validTo timestamp.

Listing 25. Excerpt from [BuybackExecutor.placeOrder](#)

```
485 // A pointer surviving the sweep is a live order. Revert to avoid placing
    overlapping orders
486 address trackedOrderAddress = lastOrderAddress;
487 if (trackedOrderAddress != address(0)) {
488     revert LiveOrderInPlace(trackedOrderAddress, lastOrderValidTo);
489 }
490
491 IStonks currentStonks = stonks;
492 uint256 stonksBalance = STETH.balanceOf(address(currentStonks));
493 uint256 sellAmount = Math.min(stonksBalance, maxAllowedOrderAmount);
494
495 if (sellAmount < minAllowedOrderAmount) {
496     revert InsufficientStonksBalance(stonksBalance,
        minAllowedOrderAmount);
497 }
498
499 // estimateTradeOutput applies the Stonks margin to the oracle estimate,
    so minBuyAmount
500 // already accounts for slippage and CoW fees.
501 uint256 minBuyAmount = currentStonks.estimateTradeOutput(sellAmount);
502
503 newOrder = currentStonks.placeOrderWithAmount(sellAmount, minBuyAmount);
504 _setLastOrderTrackingData(newOrder, block.timestamp +
    stonksOrderDurationSeconds);
```

`_sweepExpiredOrder` clears the slot only after `lastOrderValidTo`. It does not check whether the tracked order is fillable, cancelled, revoked, or completely filled, so even a fully settled order blocks new placements until its validity timestamp passes.

Listing 26. Excerpt from [BuybackExecutor._sweepExpiredOrder](#)

```
912 if (trackedOrderAddress == address(0)) {
913     return;
914 }
915 if (block.timestamp <= lastOrderValidTo) {
916     return;
917 }
```

The order's `minBuyAmount` is fixed from the placement-time oracle quote, while `Order.isValidSignature` validates fills against the current oracle quote. A caller can place an order at an in-band oracle extreme and leave it unfillable after the price moves outside the `priceTolerance` or `maxImprovement` band, but the stale order still occupies the executor's only slot until expiry.

Listing 27. Excerpt from [Order.isValidSignature](#)

```
317 if (maxImprovementBpsLocal == 0) {
318     revert PriceImprovementRejectedInStrictMode(
319         baselineBuyAmount,
320         currentEstimatedBuyAmount
321     );
322 }
```

The executor's `EMERGENCY_ROLE` can pause settlement paths, but it cannot call the live order's emergency functions as the order manager and clear the executor's pointer. Operators therefore cannot restore placement liveness before `lastOrderValidTo`, and `stETH` allocated later can remain unsold until a new order can be placed.

Exploit scenario

Alice waits for an oracle quote at a local extreme and front-runs the keeper with a permissionless `placeOrder` call. The order records a `minBuyAmount` based on that quote.

The market then moves outside the accepted band. CoW settlement cannot fill the order because `Order.isValidSignature` rejects the current quote, but Bob's keeper cannot place a replacement because `lastOrderAddress` is still set.

Carol, an emergency operator, can pause settlement but cannot cancel the order through the executor or clear the executor's slot. The buyback sell leg remains throttled until `lastOrderValidTo`, and Alice can repeat the race in later order windows.

Recommendation

Add an `EMERGENCY_ROLE`-gated function on `BuybackExecutor` that calls `emergencyCancelAndReturn` or `emergencyRevokeRelayer` on `lastOrderAddress` as the order manager, then clears `lastOrderAddress` and `lastOrderValidTo`. Consider reducing `ORDER_DURATION` and avoiding strict `maxImprovementInBps` settings that reject beneficial price movement.

Fix 1.1

The finding was fixed by adding an `EMERGENCY_ROLE`-gated function to `BuybackExecutor` that cancels the live order via `emergencyCancelAndReturn` as its manager and clears `lastOrderAddress` and `lastOrderValidTo`, restoring placement liveness before the order's validity window expires.

[Go back to Findings Summary](#)

L9: AllocationStatus enum desynchronization

LOW SEVERITY ISSUE

Impact:	Low	Likelihood:	Low
Type:	Standards violation	Discoverer:	AI
Target:	IBuybackAllocator.sol, BuybackAllocator.sol		

Description

BuybackAllocator redeclares AllocationStatus instead of using IBuybackAllocator.AllocationStatus. The two enums have diverged, because BuybackAllocator adds WindowCapReached while the interface does not.

Listing 28. Excerpt from [IBuybackAllocator.AllocationStatus](#)

```
13 enum AllocationStatus {
14     Eligible,
15     NoAvailableBudget,
16     QuoteUnavailable,
17     StEthPriceBelowMin,
18     AllocationBelowMin
19 }
```

Listing 29. Excerpt from [BuybackAllocator.AllocationStatus](#)

```
33 enum AllocationStatus {
34     // The release can proceed
35     Eligible,
36     // No budget available to spend
37     NoAvailableBudget,
38     // The oracle returned no price
39     QuoteUnavailable,
40     // The price is below the floor
41     StEthPriceBelowMin,
42     // The spendable amount is below the smallest allowed
43     AllocationBelowMin,
44     // The daily or yearly cap leaves no room
45     WindowCapReached
}
```

```
46 }
```

When `BuybackAllocator` returns or emits `AllocationStatus.WindowCapReached`, any on-chain Solidity consumer compiled against `IBuybackAllocator` will revert while decoding the return value. This happens because the consumer expects a 5-member enum (valid range `0..4`), and Solidity performs an enum bounds check during ABI decoding.

`spendable()` can return `WindowCapReached` when cap windows are exhausted.

Listing 30. Excerpt from [BuybackAllocator.spendable](#)

```
678 return (AllocationStatus.WindowCapReached, 0, 0);
```

Off-chain decoders are unaffected, because they treat enum values as raw integers.

Exploit scenario

Alice deploys an on-chain automation contract that calls `IBuybackAllocator(allocator).spendable()`. Alice uses the returned `AllocationStatus` to decide whether to trigger a buyback or record the skip reason.

When the cap window is exhausted, `BuybackAllocator` returns `WindowCapReached`. Alice's contract reverts with `Panic(0x21)` while decoding the returned enum. As a result, Alice cannot observe or act on the exhausted-cap state until the cap window resets.

Recommendation

Make `IBuybackAllocator` the single source of truth for `AllocationStatus`. Use `IBuybackAllocator.AllocationStatus` in `BuybackAllocator` (and ideally declare `BuybackAllocator is IBuybackAllocator`) instead of redeclaring the enum. Alternatively, add the missing `WindowCapReached` member to `IBuybackAllocator`

so the definitions remain byte-compatible.

Fix 1.1

The finding was fixed by having `BuybackAllocator` inherit `IBuybackAllocator` and unify the `AllocationStatus` definition, so the interface and implementation enums can no longer diverge.

[Go back to Findings Summary](#)

W1: Valuation-blind LDO buyback cadence

Impact:	Warning	Likelihood:	N/A
Type:	Front-running	Discoverer:	Human
Target:	BuybackAllocator.sol, BuybackExecutor.sol		

Description

The buyback pipeline only gates allocations based on the sell asset price via `BuybackAllocator.minStEthPriceUSD`. There is no symmetric valuation guard on the buy asset (LDO). As a result, the protocol buys LDO at the same cadence regardless of whether LDO is locally expensive or cheap.

`BuybackExecutor.placeOrder` derives `minBuyAmount` from `Stonks.estimateTradeOutput`. This enforces oracle-fair per-unit execution but does not constrain whether buying LDO at the current valuation is desirable. This reduces the capital efficiency of the buyback budget and increases MEV exposure because the buy cadence and per-call caps are predictable and can be positioned around.

Recommendation

Consider adding an LDO-side valuation guard, for example a configurable LDO max price ceiling.

Acknowledgment 1.1

The finding was acknowledged; no smart contract changes were made. According to the client, per-order pricing remains oracle-fair via the Stonks margin. Rather than introducing an on-chain LDO-side valuation guard, the client will add a section on LDO valuation oversight to the NEST handbook.

[Go back to Findings Summary](#)

W2: Chainlink phase upgrade price-read DoS

Impact:	Warning	Likelihood:	N/A
Type:	Denial of service	Discoverer:	Human
Target:	OracleRouter.sol		

Description

OracleRouter pins each token feed to the aggregator stored in FeedConfig, but `_readNormalizedPrice` re-fetches the current Chainlink Feed Registry aggregator on every read and reverts when the address changed. A routine Chainlink phase upgrade changes the registry's current aggregator, so the price path stays unavailable until the pinned config is manually resynchronized.

Listing 31. Excerpt from [OracleRouter._readNormalizedPrice](#)

```

558 address liveAggregator = registry.getFeed(baseToken_, quoteToken_);
559 uint8 liveDecimals = registry.decimals(baseToken_, quoteToken_);
560
561 if (
562     liveAggregator != feedConfig_.aggregator ||
563     liveDecimals != feedConfig_.aggregatorDecimals
564 ) {
565     revert FeedConfigOutOfSync(

```

The recovery path is permissioned, so ordinary users cannot clear the desynchronization.

Listing 32. Excerpt from [OracleRouter.syncTokenFeed](#)

```

227 function syncTokenFeed(address token_) external onlyAdminOrManager {
228     TokenConfig storage config = tokenConfig[token_];

```

For feeds bridged through ethUsdBridge, one ETH/USD aggregator upgrade can block all ETH-quoted token prices at once. During the window, buyback

order placement, fill-time order validation, LP deposits, and revenue conversion can revert; LP-mode allocation can also silently forward zero when the price read is caught.

Recommendation

Handle confirmed Chainlink aggregator upgrades without bricking the read path. Consider permissionless resynchronization, automatic acceptance of the registry's confirmed current feed, or monitoring `FeedConfirmed` events with an operational runbook for immediate `syncTokenFeed` and `syncEthUsdBridge` execution.

Acknowledgment 1.1

The finding was acknowledged; no smart contract changes were made. According to the client, the feed synchronization functions are gated and already covered by the TMC runbook, and the TMC is the managing committee responsible for exactly this operation. The client will add desynchronization alerts as an operational trigger for the committee, noting that these feeds change infrequently and can therefore be resynchronized in a timely manner. This procedure will also be documented in the NEST handbook.

[Go back to Findings Summary](#)

W3: Front-runnable precomputed Stonks address

Impact:	Warning	Likelihood:	N/A
Type:	Front-running	Discoverer:	AI
Target:	StonksFactory.sol, BuybackExecutor.sol		

Description

StonksFactory.deployStonks is permissionless and deploys a new Stonks instance via CREATE (new Stonks(...)). This makes the deployed address depend on the StonksFactory nonce.

Listing 33. Excerpt from [StonksFactory.deployStonks](#)

```

111 function deployStonks(
112     address manager_,
113     address tokenFrom_,
114     address tokenTo_,
115     address amountConverter_,
116     uint256 orderDurationInSeconds_,
117     uint256 marginInBasisPoints_,
118     uint256 priceToleranceInBasisPoints_,
119     uint256 maxImprovementInBasisPoints_,
120     bool allowPartialFill_,
121     address receiver_
122 ) external returns (address stonks) {
123     stonks = address(
124         new Stonks(
125             Stonks.InitParams(
126                 ADMIN,
127                 AGENT,
128                 manager_,
129                 tokenFrom_,
130                 tokenTo_,
131                 amountConverter_,
132                 ORDER_SAMPLE,
133                 orderDurationInSeconds_,
134                 marginInBasisPoints_,
135                 priceToleranceInBasisPoints_,
136                 maxImprovementInBasisPoints_,
137                 allowPartialFill_,
138                 receiver_

```

```
139         )
140     )
141 );
```

If governance precomputes the next CREATE address and later activates it via `setStonksAndOperatingMode(precomputed)`, an attacker can call `deployStonks` first and consume the nonce intended for governance.

`BuybackExecutor` only checks a subset of the candidate `Stonks` configuration. A `Stonks` instance deployed by a third party can still satisfy these checks while being configured differently (e.g., with an attacker-controlled `AMOUNT_CONVERTER`), causing governance to either activate an unintended instance or revert (denial of service of the intended activation).

Listing 34. Excerpt from [BuybackExecutor.setStonksAndOperatingMode](#)

```
862 address receiver = IStonks(stonks_).RECEIVER();
863
864 bool lpModeEnabled_;
865 if (receiver == address(this)) {
866     lpModeEnabled_ = true;
867 } else if (receiver != TREASURY) {
868     revert InvalidStonksReceiver(stonks_, receiver);
869 }
870
871 (address tokenFrom, address tokenTo, ) =
    IStonks(stonks_).getOrderParameters();
872 if (tokenFrom != address(STETH) || tokenTo != address(LDO)) {
873     revert InvalidStonksTokenPair(tokenFrom, tokenTo);
874 }
875
876 address stonksManager = IOwnable(stonks_).manager();
877 if (stonksManager != address(this)) {
878     revert InvalidStonksManager(stonksManager);
879 }
```

This risk is conditional on governance using a single-step workflow that relies on a precomputed nonce-based CREATE address. A two-step workflow that deploys first, observes `StonksDeployed`, and then votes to activate that exact address is not affected.

Recommendation

Deploy Stonks using CREATE2 with a user-supplied salt (optionally incorporating `msg.sender`), so governance can precompute an address that cannot be shifted by consuming the factory nonce.

If CREATE is retained, document that `deployStonks` is permissionless and nonce-dependent, and require governance to deploy first and only then vote to activate the emitted address. In this workflow, the deployer should verify the deployed instance parameters out of band before governance activation.

Acknowledgment 1.1

The finding was acknowledged; no functional smart contract changes were made beyond a clarifying code comment. According to the client, governance procedures enforce proper testing and value checking of the deployed Stonks instance before its address is set. This procedure will be documented in the NEST handbook.

[Go back to Findings Summary](#)

I1: Treasury revenue undercount from per-module rounding dust

Impact:	Info	Likelihood:	N/A
Type:	Code quality	Discoverer:	AI
Target:	StakingRevenueSource.sol		

Description

StakingRevenueSource.pushTokenRate derives treasuryFee and totalFee from StakingRouter.getStakingFeeAggregateDistribution(). It meters the DAO's fee credit by applying a single aggregate floor: $\text{treasuryShares} = \text{floor}(\text{sharesMintedAsFees} * \text{treasuryFee} / \text{totalFee})$.

In Lido's fee minting (Accounting._calculateFeeDistribution), each staking module's fee is floored individually: $\text{moduleFeeShares}_i = \text{floor}(\text{sharesMintedAsFees} * \text{moduleFee}_i / \text{totalFee})$. The treasury is then credited with the residual: $\text{treasuryShares} = \text{sharesMintedAsFees} - \text{sum}(\text{moduleFeeShares}_i)$.

Since summing multiple floored terms discards per-module rounding dust, the residual that Lido credits to the treasury is always greater than or equal to the aggregate-floored value metered by StakingRevenueSource. As a result, pendingRevenueStEth is a conservative lower bound and can undercount the DAO's actual on-chain fee credit by a few wei of shares per report (bounded by the number of fee-bearing modules).

Recommendation

If exact parity with Lido's fee minting is required, use the per-module fee distribution, floor each module's share, and compute the treasury portion as the residual $\text{sharesMintedAsFees} - \text{sum}(\text{moduleFeeShares})$. Otherwise, document that pendingRevenueStEth intentionally under-approximates the

DAO's fee credit due to per-module rounding dust.

Acknowledgment 1.1

The finding was acknowledged. The client noted that the undercount amounts to only a few wei of shares per report and is intentionally conservative. A clarifying code comment was added.

[Go back to Findings Summary](#)

I2: Staking router fee return type mismatch

Impact:	Info	Likelihood:	N/A
Type:	Code quality	Discoverer:	AI
Target:	IStakingRouter.sol, StakingRevenueSource.sol		

Description

The local IStakingRouter interface declares `getStakingFeeAggregateDistribution()` as returning uint256 fee values.

Listing 35. Excerpt from [IStakingRouter](#)

```
6 function getStakingFeeAggregateDistribution()
7     external
8     view
9     returns (uint256 modulesFee, uint256 treasuryFee, uint256
    basePrecision);
```

However, the canonical StakingRouter implementation returns the fees as uint96.

Listing 36. Excerpt from [StakingRouter](#)

```
768 function getStakingFeeAggregateDistribution()
769     public
770     view
771     returns (uint96 modulesFee, uint96 treasuryFee, uint256 basePrecision)
```

This does not currently break StakingRevenueSource because uint96 values are ABI-encoded as zero-padded 32-byte words and decode correctly into uint256.

Listing 37. Excerpt from [StakingRevenueSource](#)

```
157 (uint256 modulesFee, uint256 treasuryFee, ) =
```



```
-----  
    IStakingRouter(LIDO_LOCATOR.stakingRouter())  
158     .getStakingFeeAggregateDistribution();
```

However, the interface does not match the deployed contract ABI. This increases the risk of silent drift and can mislead reviewers and integrators about the actual fee types and bounds.

Recommendation

Align the local `IStakingRouter` return types with the canonical `StakingRouter` ABI (`uint96 modulesFee`, `uint96 treasuryFee`, `uint256 basePrecision`). Alternatively, document that widening to `uint256` is intentional and safe, and pin the interface to an upstream source to prevent future drift.

Fix 1.1

The `IStakingRouter` interface was updated.

[Go back to Findings Summary](#)

I3: Ignored slippage floors on full LP withdrawals

Impact:	Info	Likelihood:	N/A
Type:	Logic error	Discoverer:	AI
Target:	BuybackExecutor.sol		

Description

`removeLiquidityAndRecoverToTreasury` forwards `minLdoAmount_` and `minWstEthAmount_` to Curve's two-sided `remove_liquidity` call and does not check the returned amounts itself.

Listing 38. Excerpt from [BuybackExecutor.removeLiquidityAndRecoverToTreasury](#)

```
434 uint256[2] memory withdrawn = CURVE_POOL_AND_TOKEN.remove_liquidity(  
435     lpAmount_,  
436     [minLdoAmount_, minWstEthAmount_]  
437 );
```

TwoCryptoNG enforces `_min_amounts` only for partial LP redemptions. When `_amount == totalSupply`, it returns the raw pool reserves without applying the minimum amount checks. Therefore, if the executor is the sole LP and a manager withdraws the full position, the documented slippage floors are inert and the call can succeed even when the returned token amounts are below the provided floors.

This has no value impact because a full two-sided redemption withdraws all reserves to TREASURY; it only makes the min-out parameters misleading on this path.

Recommendation

Check the returned `withdrawn` amounts against `minLdoAmount_` and `minWstEthAmount_` after `remove_liquidity` returns. Alternatively, document that

the min-out parameters are enforced only for partial LP withdrawals.

Fix 1.1

The finding was fixed by documenting that the min-out parameters are enforced only for partial LP withdrawals.

[Go back to Findings Summary](#)

I4: Missing interface inheritance

Impact:	Info	Likelihood:	N/A
Type:	Code quality	Discoverer:	AI
Target:	BuybackAllocator.sol, Order.sol, Ownable.sol		

Description

BuybackAllocator, Order, and Ownable define members that are also declared in their matching I* interfaces, but do not inherit the interface. As a result, the compiler does not enforce that the implementation stays consistent with the intended external surface, and edits to either side can silently drift. This already materialized once when an enum type diverged between BuybackAllocator and IBuybackAllocator.

Listing 39. Excerpt from [BuybackAllocator](#)

```
23 contract BuybackAllocator is AssetRecovererACL {
```

Listing 40. Excerpt from [Order](#)

```
27 contract Order is IERC1271, AssetRecoverer {
```

Listing 41. Excerpt from [Ownable](#)

```
11 contract Ownable {
```

Recommendation

Have each implementation inherit its authoritative I* interface. Reference interface-defined types instead of redeclaring them.

Fix 1.1

BuybackAllocator, which is in scope, was updated to inherit its IBuybackAllocator interface. The interface inheritance for Order and Ownable was not addressed.

[Go back to Findings Summary](#)

Report Revision 1.1

Revision Team

Revision team is the same as in [Report Revision 1.0](#).

Report Revision 1.2

Revision Team

Revision team is the same as in [Report Revision 1.0](#).

Appendix A: How to cite

Cite this document as:

[ack3](#), Security Review | Lido: NEST, July 24, 2026.



Thank you.

COMPANY	ACK3 LTD
WEB	ack3.ai
X	@ack3ai
MAIL	hello@ack3.ai